PROCEEDINGS OF SCIENCE

ChatSOC: A Large Language Model Powered Autonomous Agent for Security Operations Center

Jiarong Wang[a,*], **Caiqiu Zhou**[✉,a,b], **Yang Yi**[a,c], **Qianran Sun**[d], **Tian Yan**[a], **Fazhi Qi**[a,b]

[a] *Computing Center, Institute of High Energy Physics, Chinese Academy of Sciences, Beijing 100049, P.R.China*

[b] *Spallation Neutron Source Science Center, Dongguan 523803, P.R.China*

[c] *University of Chinese Academy of Sciences, Beijing 100049, P.R.China*

[d] *School of Advanced Interdisciplinary Sciences,University of Chinese Academy of Sciences, Beijing 100049, P.R.China*

E-mail: wangjr@ihep.ac.cn,zhoucq@ihep.ac.cn

To defend against numerous cyberattacks, the Security Operations Center at the Institute of High Energy Physics (IHEP SOC) was established in 2021. IHEP SOC implemented a standardized procedure for invoking several security tools to execute automated response tasks. This standardized process utilizes a predefined set of tools to ensure that the IHEP SOC maintains minute-level response times, with a remarkably low false block rate of one in ten thousand. However, the fixed process of tool invoking is often challenging to update and manage, and it typically addresses isolated security operational tasks. To address this issue, this paper introduces ChatSOC, an autonomous agent for network security operations empowered by an LLM, capable of completing various operational tasks: identification, protection, detection, and response. ChatSOC streamlines these operations by dynamically planning and executing tasks without relying on fixed processes for tool invocation. Through its deployment at the Institute of High Energy Physics (IHEP), ChatSOC has proven highly effective, enhancing security operations by automatically planning and invoking different tools.

International Symposium on Grids and Clouds (ISGC2025)
16 -21 March 2025
Academia Sinica Grid Computing Centre (ASGC), Taipei, Taiwan

[*] Speaker

© Copyright owned by the author(s) under the terms of the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0)
All rights for text and data mining, AI training, and similar technologies for commercial purposes, are reserved.
ISSN 1824-8039 . Published by SISSA Medialab.

PoS(ISGC2025)009

1. Introduction

The IHEP network faces a multitude of cybersecurity threats, including intrusions, malware, cryptocurrency mining, ransomware, resource abuse, and copyright violations. To bolster its defense mechanisms, the Institute established the Security Operations Center (SOC) in 2021. The IHEP SOC has implemented a standardized procedure for invoking security tools to execute automated response tasks. Within this procedure, the SOC utilizes a threat detection tool to identify potential threats, which are then cross-referenced against a whitelist. If a detected entity is not on the whitelist, it is classified as malicious and subsequently added to the blacklist. Finally, the entity is blocked using a dedicated blocking tool. This standardized process ensures that the IHEP SOC maintains minute-level response times, with a remarkably low false block rate of one in ten thousand.

However, this standardized process relies on a predefined, fixed workflow to invoke various security tools. These fixed processes are often challenging to update and manage, and they typically address isolated security operational tasks. This rigidity raises a key challenge: how can we automatically generate workflows to complete diverse security operational tasks in a more dynamic manner?

To address this issue, this paper presents ChatSOC, an autonomous agent powered by a large language model designed for security operations centers. ChatSOC features an interactive web interface that allows users to input security operational tasks. Subsequently, ChatSOC analyzes these tasks, decomposes them into a sequence of subtasks, defines their dependencies and execution order, and maps each subtask to an appropriate security tool for execution. Finally, the outputs of these tools are summarized and returned to the interactive web interface.

The main contributions of this paper are as follows:

- We propose an LLM-powered autonomous agent that enables end-to-end, multitask automation, significantly improving security operations efficiency at IHEP.
- We design a novel task planning mechanism that enables the agent to autonomously select and orchestrate multiple security tools to complete complex operations without manual intervention.
- We implement and deploy ChatSOC as a practical tool at IHEP, validating its effectiveness in a real-world setting and providing operators with an intuitive web-based interface.

2. Related works

In recent years, the advanced reasoning capabilities of large language models (LLMs) [1-5] have driven significant advancements in intelligent agent technology. LLM-based agents have emerged as pivotal tools in task automation and human assistance, achieving breakthroughs across various domains.

Regarding system architecture, two primary approaches have emerged: single-agent and multi-agent systems. Single-agent systems, such as LangChain [6] and HuggingGPT [7], focus on independent task processing, while multi-agent frameworks like AutoGen [8] and CAMEL [9] address complex problems through distributed collaboration. Research indicates that employing more powerful foundation models like GPT-4 [10] and PaLM [11], along with designing

PoS(ISGC2025)009

sophisticated multi-LLM interaction mechanisms, can significantly enhance overall agent performance.

In terms of reasoning methodologies, researchers have developed a spectrum of methodologies, ranging from foundational to advanced approaches. The evolution has progressed from the initial Chain-of-Thought (CoT) [12] step-by-step reasoning to the "Think-Act" mechanism of the ReAct [13] framework, and further to the closed-loop reflection process of the Reflexion [14] method, continuously strengthening agents' decision-making capabilities. Particularly, the Tree-of-Thoughts (ToT) [15] approach enables multi-step prospective planning, while multi-agent discussion [16] systems optimize decision quality through collective intelligence, enabling agents to tackle increasingly complex real-world scenarios.

However, these general-purpose LLM-based agents cannot be directly applied to the domain of network security operations. This is because the invocation of specialized security tools requires domain-specific design and adaptation to accommodate the diverse operational tasks and practical challenges inherent to a real-world environment like the IHEP SOC.

3. Methodology

3.1 Problem definition

To autonomously analyze and execute different network security operational tasks, we first formalize a network security operation problem P as a tuple: $P = \langle T, C \rangle$, where T represents a task and C represents the context. The task T defines the specific autonomous operation to be performed, categorized into four types: identify, protect, detect, and response. Each task type is associated with specific network security activities. For example, the "identify" type encompasses identifying vulnerabilities and weak passwords; the "protect" type involves configuring network security policies; the "detect" type includes identifying network attacks; and the "response" type consists of blocking malicious activities and reporting incidents.

The context C is further formalized as a tuple: $C = \langle I, E \rangle$, where I is the background information of the organization, and E represents the security operational engineers. The background information I encompasses the information security status, IT infrastructure, basic security capabilities, and common internal abbreviations. The information security status includes the network domains, public IP address ranges, and internal office IP address segments. The IT infrastructure covers the organization's industry, scale, and network architecture. Basic security capabilities refer to the deployed security products. Common internal abbreviations include Security Operations Center (SOC), Web Application Firewall (WAF), and Demilitarized Zone (DMZ). The security operational engineers, E, consist of analysts, operators, and responders responsible for security operations.

3.2 System framework

The ChatSOC architecture, as illustrated in Figure 1, comprises two primary components:

(1) User Interface (UI): The frontend and backend components that facilitate user interaction with the automated security operations module.

(2) Automatic Security Operations: A module that integrates scheduling, memory, and security tools to execute security operations.

PoS(ISGC2025)009

ChatSOC provides a robust interface for seamless communication between users and the operations module. Upon receiving user instructions, ChatSOC formulates a plan and executes it by utilizing the appropriate security tools.

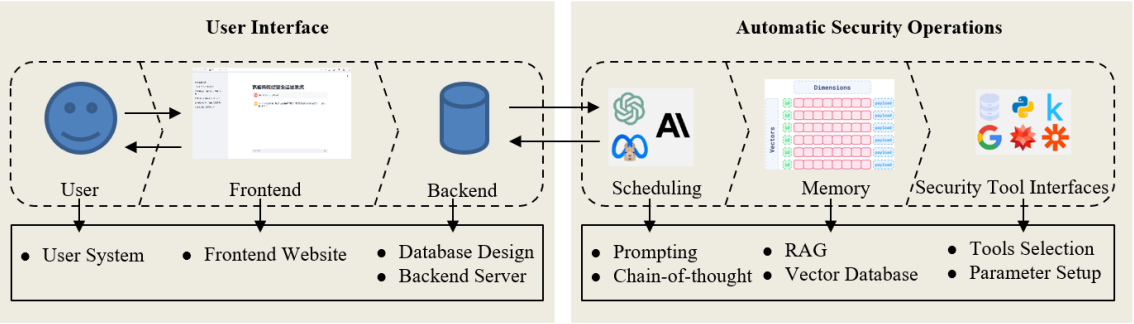


Figure 1: Framework of ChatSOC

User Interface. We designed a user-friendly and highly functional User Interface for communication between users and the operations module. The User Interface mainly consists of user authentication, FAQ guidance, dialogue interaction, and response display. Our objective is for ChatSOC to be not only user-friendly but also to offer a highly responsive interactive experience and lightweight deployment.

Automatic Security Operations. In our design, the Automatic Security Operations module consists of three essential components: a scheduling module, memory databases, and security tool interfaces. In the scheduling module, we employ prompting methods based on chain-of-thought techniques. This enables ChatSOC to follow a sequential process of thought, action, and observation in each interaction turn. The memory databases provide knowledge supplementation for ChatSOC, such as the organization's background information. The tool interface includes parsers that translate natural-language text into executable actions, such as code generation or API calls. Subsequently, these actions are executed, and the results are returned to the User Interface.

3.3 Implementation

3.3.1 User Interface

The user interface is developed using the Streamlit framework, offering an optimal user interaction experience with lightweight deployment. The core functional modules are user authentication, FAQ guidance, interactive dialogue, and model response display.

We have implemented a comprehensive user login and registration system for secure access control. Users can register with a username and password. The Streamlit-based interface collects credentials via forms, and the backend stores account information in an SQLite database, preventing duplicate registrations. The system authenticates users by verifying their credentials against the database. Upon successful verification, users are automatically redirected to the main interface; otherwise, an error message is displayed.

After successful login, users access the main interface of the "IHEP Security Operations System," which comprises the following components:

Q&A Interaction Zone: Showcases the natural-language Q&A between the user and the automated system. Users submit instructions and receive real-time responses.

Conversation History: Maintains the dialogue context by categorizing messages into "User" and "Assistant" roles, facilitating seamless multi-turn conversations.

PoS(ISGC2025)009

FAQ Guidance: A sidebar provides quick-access prompts for common operations, such as "IP Block Check" and "Abnormal Login Handling," allowing users to reference solutions or initiate queries instantly.

Logout Functionality: Users can securely end their session with a single click, which clears all login credentials and cached data.

The system utilizes real-time streaming output to dynamically display model responses, significantly enhancing the perceived responsiveness and smoothness of the interaction.

3.3.2 Automatic Security Operations

The Automatic Security Operations module consists of a scheduling module, memory databases, and security tool interfaces.

The scheduling module receives instructions from users and formulates task plans. We have adopted chain-of-thought techniques to create prompts for task planning using large language models (LLMs). This allows LLMs to analyze user instruction T and break it down into a set of subtasks $T = \{t_1, t_2, \ldots, t_m\}$ with specific dependencies and execution orders. Each subtask is optimized for execution using a single security tool. The core concept of the chain-of-thought technique entails dividing a task into multiple steps and solving it incrementally through a cyclical process comprising three phases: thought, action, and observation. Specifically, the "thought" phase involves identifying appropriate actions based on parsed user input and the current state of information retrieval. The "action" phase entails selecting and invoking the appropriate security tools with the necessary parameters. The "observation" phase consists of evaluating the results returned by the invoked tools to ascertain whether the user's instructions have been fully addressed. If the results from the security tools do not satisfactorily resolve the task, the iterative process of these three phases continues to ensure the accuracy of the answer. This iteration process is depicted in Figure 2.

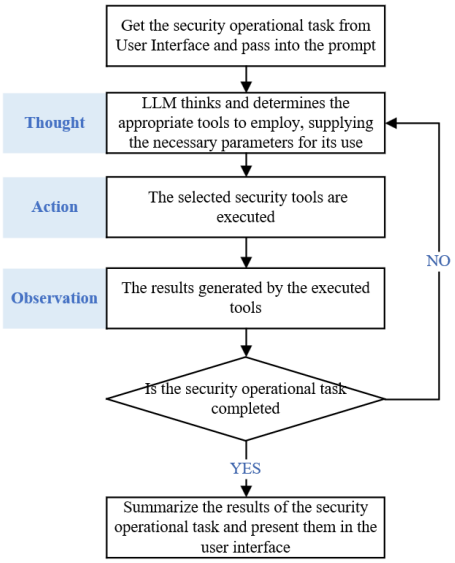


Figure 2: Security operational task scheduling

The memory database offers knowledge supplementation for ChatSOC. When automatic security operations encounters a subtask beyond its knowledge scope, it first checks if the required knowledge is available in the memory database. If not, it uses the security tools to perform the subtask. To retrieve knowledge from the database, we use retrieval-augmented generation (RAG)

PoS(ISGC2025)009

technology. This technology can obtain relevant knowledge from the database and integrate it into the generation process of the LLM. RAG is implemented using a vector database, which can quickly retrieve the most relevant information snippets by converting input text into vectors and calculating the similarity with vectors stored in the database. When the ChatSOC is processing a subtask, it first retrieves the historical subtasks and their results from the vector database, which enhances the ChatSOC's efficiency.

The tool interface includes parsers that can translate natural language text into executable actions. We define the tool interfaces as structured variables, which can be denoted as:

$$tools = \{n_i, d_i, p_i, r_i\}_{i=1}^{K} \quad (1)$$

where K is the number of available security tools, n is the name of the security tool, d is the function description of the security tool, p represents the parameters required to execute the security tool, and r provides the parameter format required when the LLM passes it to the corresponding security tool. The ChatSOC matches subtasks with suitable tools from the tool inventory based on the similarity between the subtasks and tool descriptions. Once the most appropriate tool for each subtask is selected, the LLM can extract the parameters based on p and r in the structured tool variables. After tool selection and parameter setup, the tool can be executed.

4. Functionality

Based on the security operational requirements of IHEP, we classify the security operational tasks into four types. Each type and its corresponding security operational tasks can be found in Table 1.

Table 1. Security operational tasks

Security operational type	Security operational task
Protect	Is the specific security device (WAF, firewall, NIDS, HIDS) running properly
	Please check if the specific website has been included in WAF protection
	Please check if the specific host has deployed a host monitoring system
	Which servers have opened the specific port to the external network
	Please check if the specific IP is blocked
Detect	Is there the specific attack (SQL injection, brute force, phishing email, APT) in the network
Identify	Perform vulnerability scanning on the specific website or host
	Please search for websites or hosts with high-risk vulnerabilities
	Is there a weak password account in the network
Response	Please send an email to the administrator to remind them to fix high-risk vulnerabilities
	Please send an email to users with weak password accounts to remind them to change their passwords

PoS(ISGC2025)009

	Please send an email to the specific email address to notify the processing device that it has been infected and to perform antivirus treatment.
	Block the specific IP address
	Generate the incident reporting

The main function of ChatSOC is to complete the security operational tasks listed in Table 1. As an example, Figure 3 shows how ChatSOC handles the task "Check if a specific IP is blocked." When a user inputs the security operational instruction "Check the block status of 1.0.146.146" in the User Interface, the automatic security operations module will select and invoke the blocking tool to query the status for the specific IP. The returned information is then displayed in the User Interface.



Figure 3: The display of the function for ChatSOC

5. Conclusions

To address the inefficiencies associated with the use of multiple discrete security tools in the daily security operations at IHEP, we propose ChatSOC, an autonomous agent designed for security operations centers and powered by a large language model. ChatSOC consists of a user interface and an automatic security operations module designed to complete security tasks. Specifically, it leverages the large language model (LLM) to select and invoke multiple security tools without relying on a preset, fixed process for tool invocation. ChatSOC is applied at IHEP and improves operational efficiency from two perspectives: 1) the operational time decreases from several hours to minutes; 2) The reliance on security professionals' experience is reduced, based on ChatSOC's performance in tool selection and invocation. In the future, we will conduct research on automated API integration. This involves dynamically inferring tool descriptions and parameter requirements from API documentation.

PoS(ISGC2025)009

Acknowledgments

This work is supported by the National Key Research and Development Program of China (No. 2023YFC2206402 and No. 2023YFC2605703) and by the Cybersecurity Joint Defense System for Large Scientific Facilities Construction Project of Chinese Academy of Sciences and by the Innovative Project of the Institute of High Energy Physics of Chinese Academy of Sciences (No. E55456U2).

References

[1] Y. Zhang, S. Mao, T. Ge, X. Wang, A. Wynter, Y. Xia, W. Wu, T. Song, M. Lan, and F. Wei, *Llm as a mastermind: A survey of strategic reasoning with large language models*, arXiv preprint arXiv:2404.01230, 2024c.

[2] S. Qiao, Y. Ou, N. Zhang, X. Chen, Y. Yao, S. Deng, C. Tan, F. Huang, and H. Chen, *Reasoning with language model prompting: A survey*, arXiv preprint arXiv:2212.09597, 2022.

[3] L. Fan, W. Hua, L. Li, H. Ling, Y. Zhang, and L. Hemphill, *Nphardeval: Dynamic benchmark on reasoning ability of large language models via complexity classes*, arXiv preprint arXiv:2312.14890, 2023.

[4] L. Fan,W. Hua, X. Li, K. Zhu, M. Jin, L. Li, H. Ling, J. Chi, J. Wang, X. Ma, *Nphardeval4v: A dynamic reasoning benchmark of multimodal large language models*, arXiv preprint arXiv:2403.01777, 2024.

[5] M. Jin, Q. Yu, H. Zhao, W. Hua, Y. Meng, Y. Zhang, M. Du, *The impact of reasoning step length on large language models*, arXiv preprint arXiv:2401.04925, 2024.

[6] O. Topsakal and T. Akinci, *Creating large language model applications utilizing langchain: A primer on developing llm apps fast*, In International Conference on Applied Engineering and Natural Sciences, volume 1, pp. 1050–1056, 2023.

[7] Y. Shen, K. Song, X. Tan, D. Li,W. Lu, and Y. Zhuang, *Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face*, Advances in Neural Information Processing Systems, 36, 2024.

[8] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S.Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang, *Autogen: Enabling next-gen llm applications via multiagent conversation framework*, aXiv preprint arXiv:2308.08155, 2023.

[9] G.Li, H.Hammoud, H.Itani, D.Khizbullin, and B.Ghanem, *Camel: Communicative agents for "mind" exploration of large language model society*, Advances in Neural Information Processing Systems, 36:51991–52008, 2023.

[10] OpenAI, "Gpt-4 technical report," 2023.

[11] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. Won Chung, C. Sutton, S. Gehrmann, *Palm: Scaling language modeling with pathways*, arXiv preprint arXiv:2204.02311, 2022.

[12] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V Le, D. Zhou, *Chain-of-thought prompting elicits reasoning in large language models*, Advances in neural information processing systems, 35:24824–24837, 2022.

[13] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, *React: Synergizing reasoning and acting in language models*, arXiv preprint arXiv:2210.03629, 2022.

[14] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, *Reflexion: Language agents with verbal reinforcement learning*, Advances in Neural Information Processing Systems, 36, 2024.

PoS(ISGC2025)009

[15] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, *Tree of thoughts: Deliberate problem solving with large language models*, Advances in Neural Information Processing Systems, 36, 2024.

[16] C. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu, and Z. Liu, *Chateval: Towards better llm-based evaluators through multi-agent debate*, arXiv preprint arXiv:2308.07201, 2023.

PoS(ISGC2025)009